

AMA-KBQA: An Adaptable Multi-Agent Framework for Generalizable Knowledge Base Question Answering

Yannic Duan¹, Maximilian Klattig¹, Maurice Bardenhagen¹, Yuyang Li¹ and Tobias Käfer¹

¹Karlsruhe Institute of Technology (KIT), Institute of Applied Informatics and Formal Description Methods (AIFB), Karlsruhe, Germany

Abstract

We present **AMA-KBQA**, a multi-agent framework for retrieval-augmented generation (RAG) over knowledge graphs that decouples reasoning by large language models (LLMs) from graph retrieval through abstract agents, configuration-driven adapters, and Model Context Protocol (MCP) tool servers. The system requires *no model training*: onboarding a new knowledge graph reduces to writing SPARQL wrappers behind an abstract tool interface, and adopting a stronger LLM is a configuration change. A journal-based scratchpad keeps reasoning state token-efficient, and a Lessons-Learned loop distills high-quality traces into few-shot examples. Across KQAPro and SciQA, the same reasoning core generalises over a reified encyclopedic graph and a hierarchical scientific graph, reaching up to 84% accuracy.

Keywords

KBQA, SPARQL, Large Language Models, Model Context Protocol, Knowledge Graphs

Demo <https://purl.archive.org/ama-kbqa-demo> **Code+Video** <https://github.com/yannicd03/AMA-KBQA>

1. Introduction

Knowledge Base Question Answering (KBQA) is the task of answering natural-language questions over a structured knowledge base, typically a Resource Description Framework (RDF) graph queried via SPARQL. KBQA makes curated graph data accessible to non-experts, and large language models (LLMs) have become the default instrument for it. Yet LLMs alone suffer from knowledge cutoff, hallucination, and poor multi-hop reasoning over structured data [1]. Text-to-SPARQL semantic parsing [2] and retrieval-augmented generation (RAG) [3] ground the model in the graph, but bind the resulting system tightly to one graph’s schema and vocabulary, and state-of-the-art KBQA systems additionally depend on dataset-specific training, from fine-tuned query generators [4] to trained question classifiers [5]. Both forms of coupling are costly: porting to a new graph or adopting a stronger model means re-engineering or retraining the pipeline.

This rigidity is particularly painful because deployed knowledge graphs are not static. Industry-scale graphs evolve continuously as new domains are onboarded, schemas are refac-

SEMANTiCS 2026: 22nd International Conference on Semantic Systems, September 15–17, 2026, Ghent, Belgium

EMAIL: yannic.duan@student.kit.edu (Y. Duan); maximilian.klattig@student.kit.edu (M. Klattig);

maurice.bardenhagen@student.kit.edu (M. Bardenhagen); yuyang.li@kit.edu (Y. Li); tobias.kaefer@kit.edu

(T. Käfer)



© 2026 Copyright for this paper by its authors. Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).

 CEUR Workshop Proceedings (CEUR-WS.org)

tored, and data sources are added, replaced, or deprecated [6]; rebuilding the pipeline for every such change is impractical. What is needed is a framework that treats both the knowledge graph (KG) and the language model as swappable components, adapted by configuration rather than re-implementation.

We introduce **AMA-KBQA**, a multi-agent framework built around exactly this requirement. It counters coupling to the graph with an abstract tool interface: the reasoning core only ever calls typed graph operations, which each graph implements as deterministic SPARQL wrappers that insert the tool’s parameters into a valid query template instead of letting the LLM write SPARQL freely. The backbone LLM itself is a configuration choice. And it requires no dataset-specific training: a question is answered by (i) deterministic *pre-processing* (question classification and entity extraction), (ii) *agentic exploration* via typed tool calls over SPARQL and vector search, and (iii) *grounded synthesis* over a scratchpad of verified facts rather than free LLM generation. On top of these three phases, a Lessons-Learned loop distills highly rated traces into few-shot examples, so the system improves over time without fine-tuning.

2. Related Work

Text-to-SPARQL systems translate a question directly into a query: classical semantic parsers learn the mapping from question-answer pairs [2], neural approaches treat SPARQL as a target language for sequence-to-sequence models [7, 4], and recent work prompts LLMs to generate SPARQL directly, still with limited reliability [8]. All of these couple the system to one graph’s vocabulary and structure, so a schema change degrades the learned mapping. Interactive-KBQA [5] instead reframes KBQA as a multi-turn control loop over atomic graph APIs in the spirit of ReAct [9], paired with a trained BERT question classifier. We adopt this agentic stance and extend it for deployability: a configuration-driven adapter layer ports the same reasoning core to a new knowledge graph without rewrites or training, the typed tools are exposed via the Model Context Protocol (MCP) [10], an orchestrator federates questions across sub-agents, and a Lessons-Learned loop improves the system from its own traces, following Synapse’s trajectory-as-exemplar approach [11].

3. Architecture

Federated multi-source design. AMA-KBQA centres on a shared reasoning core whose sub-agents are instances of the same template and differ only in their knowledge-graph adapter. All data access goes through typed MCP tools for entity discovery, graph navigation, and schema introspection, backed by a triple store and a vector store. Since each sub-agent is self-contained, an *orchestrator* can drive several of them (Fig. 1, right): it probes each data source’s vector store with the query’s top entities, commits to a route based on this evidence, dispatches the question to the relevant sub-agents, and merges their grounded scratchpads into a single cross-source response.

Agent lifecycle. Each sub-agent runs the same three-phase lifecycle (Fig. 1, left). A deterministic pre-agent hook classifies the question by type and extracts candidate entities, injecting a

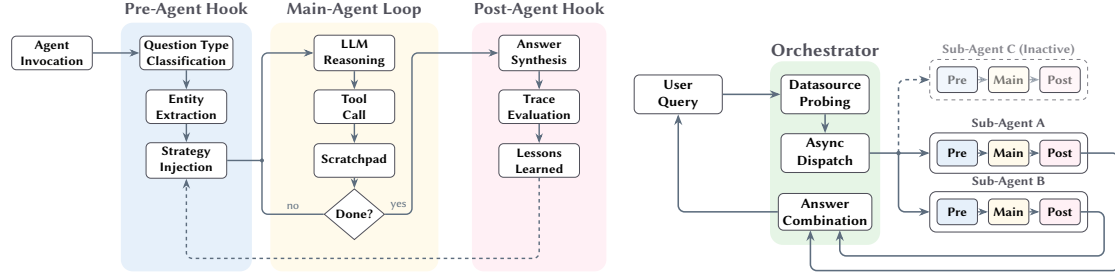


Figure 1: Left: agent lifecycle, with deterministic pre-processing, an iterative tool-calling main loop over a scratchpad, and post-hoc synthesis, trace evaluation, and lesson distillation. Right: the multi-agent orchestrator dispatches the question to the sub-agents matched by the routing probe (each an instance of the left lifecycle) and merges their grounded answers; unmatched sub-agents stay inactive.

matching strategy document and type-specific few-shot examples into the system prompt. The main loop then iteratively calls typed tools and writes its findings to a structured scratchpad [12] of visited nodes, discovered values, and verified facts; deterministic comparison and verification tools offload numeric and boolean logic from the language model, and a loop detector injects recovery guidance when the agent stalls. Finally, a post-agent hook synthesises the answer solely from the scratchpad’s verified facts, and a judging agent scores the trace so that good runs become few-shot examples for future queries.

Access control by design. Since every knowledge-graph interaction passes through deterministic SPARQL wrappers, access policy can be enforced at a single, auditable boundary: each typed tool can inject named-graph filters or row-level predicates derived from the caller’s identity before a query reaches the triple store, which is more verifiable than prompt-based restrictions; enforcing such policies is future work.

KG onboarding. A hierarchy of dataclasses declares everything the reasoning core needs to know about a graph: Uniform Resource Identifier (URI) prefixes, endpoint and named-graph settings, reification and temporal-data flags, and vector-collection bindings. Onboarding a new KG follows four steps. (1) The RDF dump is loaded into the triple store under a dedicated named graph. (2) Entity labels and relation names are indexed into two vector collections, powering fuzzy entry-point discovery and the routing probe. (3) `BaseKGAdapter` is subclassed to return the populated configuration, and the graph is exposed through an MCP server that implements the abstract tool interface as endpoint-specific SPARQL templates. (4) The new sub-agent, which declares only its adapter and MCP-server path, is registered with the orchestrator. Both reference integrations are a few hundred lines of largely declarative code; an unseen graph can typically be onboarded in a few hours.

Evaluation excerpt. Table 1 reports representative results with an open-weights backbone: 84% on the Wikidata-derived KQAPro benchmark [13] and, through its own adapter without core changes, 73% on the SciQA benchmark [14] over the Open Research Knowledge Graph (ORKG).

Table 1

Evaluation excerpt: 100 sampled questions per dataset, per-question averages. At public per-token rates a question stays below \$0.04.

Dataset	LLM	Accuracy	Avg. time	Avg. tokens
KQAPro	Gemma4-31B	84%	30.8 s	69.4 k
SciQA	Gemma4-31B	73%	37.5 s	129.7 k

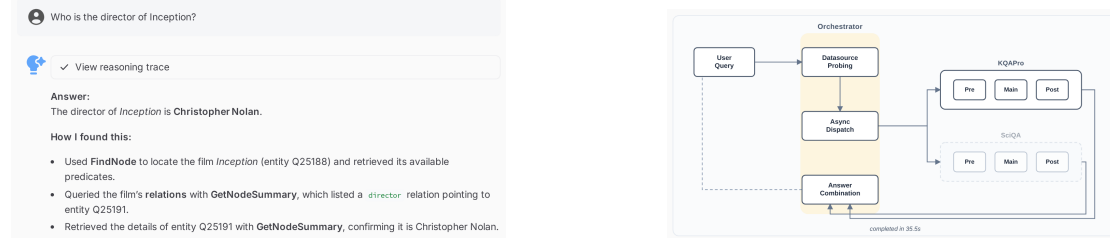


Figure 2: The demo answering a question via the Orchestrator. Left: the grounded answer cites the tool calls behind each step. Right: the live swarm view; the probe matched KQAPro, so only that sub-agent animates while SciQA stays inactive.

4. Demonstration

At our demo booth, attendees pick an agent (the Orchestrator by default, or a KQAPro/SciQA specialist with stateful multi-turn follow-ups) and a backbone LLM. While the agent runs, the UI animates the lifecycle of Fig. 1 (left) live, and in Orchestrator mode the swarm view of Fig. 2: stages light up as their trace spans open, and the routing probe’s per-source evidence feeds an explicit `select_agent` decision. Once the answer arrives (Fig. 2, left), a Trace tab exposes the full trace as a span tree, down to individual LLM messages, tool calls, and the raw SPARQL issued.

5. Conclusion

We showcased AMA-KBQA, a functional prototype of a training-free multi-agent system for KBQA over heterogeneous RDF graphs. A single reasoning core with per-graph MCP SPARQL-wrapper tools reaches competitive accuracy on KQAPro and ports to SciQA without any change to the core (Table 1), at practical runtimes and costs. Accuracy on SciQA still trails KQAPro’s, automated onboarding remains open, and the wrapper boundary is designed for per-caller access control that we have yet to enforce; we see these as promising directions towards a portable, model-agnostic KBQA ecosystem.

Acknowledgments

This work was carried out in the Seminar Knowledge Discovery at KIT’s AIFB institute and supported by the Deutsche Forschungsgemeinschaft (DFG) under Research Unit FOR 5339

(Project No.459291153).

Declaration on Generative AI

The authors used Claude for grammar and spelling checks, paraphrasing, and implementation assistance; the LLMs inside AMA-KBQA are themselves the object of study. Generative AI was not used to conceive the research idea, make architectural decisions, formulate the evaluation, or draw conclusions. The authors reviewed and edited all code and content and take full responsibility for it.

References

- [1] G. Agrawal, T. Kumarage, Z. Alghamdi, H. Liu, Can knowledge graphs reduce hallucinations in LLMs?: A survey, in: Proc. of NAACL, 2024.
- [2] J. Berant, A. Chou, R. Frostig, P. Liang, Semantic parsing on Freebase from question-answer pairs, in: Proc. of EMNLP, 2013.
- [3] P. Lewis, et al., Retrieval-augmented generation for knowledge-intensive NLP tasks, in: Proc. of NeurIPS, 2020.
- [4] M. R. A. H. Rony, et al., SGPT: A generative approach for SPARQL query generation from natural language questions, IEEE Access 10 (2022).
- [5] G. Xiong, J. Bao, W. Zhao, Interactive-KBQA: Multi-turn interactions for knowledge base question answering with large language models, in: Proc. of ACL, 2024.
- [6] N. Noy, Y. Gao, A. Jain, A. Narayanan, A. Patterson, J. Taylor, Industry-scale knowledge graphs: Lessons and challenges, Commun. ACM 62 (2019).
- [7] T. Soru, et al., SPARQL as a foreign language, in: Proc. of SEMANTiCS Posters & Demos, 2017.
- [8] L.-P. Meyer, J. Frey, N. Arndt, Assessing SPARQL capabilities of large language models, in: Proc. of NLP4KGC at SEMANTiCS, 2024.
- [9] S. Yao, et al., ReAct: Synergizing reasoning and acting in language models, in: Proc. of ICLR, 2023.
- [10] Anthropic, Model Context Protocol specification, 2024. URL: <https://modelcontextprotocol.io>.
- [11] L. Zheng, R. Wang, X. Wang, B. An, Synapse: Trajectory-as-exemplar prompting with memory for computer control, in: Proc. of ICLR, 2024.
- [12] M. Nye, et al., Show your work: Scratchpads for intermediate computation with language models, 2021. URL: <https://arxiv.org/abs/2112.00114>.
- [13] S. Cao, et al., KQA Pro: A dataset with explicit compositional programs for complex question answering over knowledge base, in: Proc. of ACL, 2022.
- [14] S. Auer, et al., The SciQA scientific question answering benchmark for scholarly knowledge, Scientific Reports 13 (2023).